Algorithmes et Pensée Computationnelle

Programmation orientée objet - Exercices avancés

Le but de cette séance est de découvrir et de mettre en pratique un paradigme de programmation largement utilisé : la programmation orientée objet (POO). Ce paradigme repose sur la création d'objets qui regroupent des données et des comportements, et qui peuvent interagir entre eux. Au fil des exercices, vous apprendrez à manipuler des objets et à maîtriser les notions fondamentales de classe, d'attribut, de méthode et d'encapsulation. À l'issue de cette séance, vous serez capables de concevoir des programmes mieux structurés et plus faciles à maintenir.

Les exercices de cette séance seront réalisés en Java et en Python.

Les durées indiquées (🕓) sont données à titre indicatif.

1 Interaction entre plusieurs instances d'une même classe - Java (30 minutes)

Dans cette section, nous allons simuler un jeu de combat entre deux protagonistes représentant des instances d'une classe `Fighter` que nous allons créer. Chaque `Fighter` aura des attributs qui le définissent. Ces attributs sont :

- `nom`:*(String)* chaque combattant sera identifié par un nom unique.
- `health`:*(int)* représentant le nombre de points de vie d'un combattant. Il contient des valeurs comprises entre 0 et 10. À l'instanciation de l'objet, le combattant a 10 points de vie par défaut.
- `attaque`:*(int)* représentant une valeur qui sera utilisée pour calculer le nombre de points de dégâts infligés à l'adversaire.
- `défense`:*(int)* représentant une valeur qui sera utilisée pour calculer le nombre de points de dégâts reçus.

Deux attributs de classe seront également utilisés :

- `instances` : Liste comprenant les combattants qui ont été instanciés et qui sont toujours en vie.
- `attackModifier` : Dictionnaire comportant 3 types d'attaques, chacune modifiant les dégâts qui vont être infligés. Les trois types d'attaques sont `poing, pied et tête` modifiant respectivement l'attaque par 1, 2, 3.

Le but de cette partie est d'étudier les interactions entre deux instances d'une même classe. Cette classe se présentera sous la forme d'un `Fighter`. Chaque instance de la classe `Fighter` pourra attaquer les autres instances.
Vous devrez compléter les 4 méthodes suivantes :

1. `isAlive()`
2. `checkDead()`
3. `checkHealth()`
4. `attack(String type, Fighter other)`

Voici le squelette du code (à télécharger sur Moodle) :

```
1  import java.util.HashMap;
2  import java.util.List;
3  import java.util.ArrayList;
4  import java.util.Map;
5
6  public class Fighter {
7      private String name;
8      private int health;
```

```
9       private int attack;
10      private int defense;
11      private static List<Fighter> instances = new ArrayList<Fighter>();
12      private static HashMap<String, Integer> attackModifier = new
        HashMap<String, Integer>(
13              Map.of("poing", 1, "pied", 2, "tete", 3));
14
15      public Fighter(String name, int health, int attack, int defense) {
16          this.name = name;
17          this.health = health;
18          this.attack = attack;
19          this.defense = defense;
20          instances.add(this);
21      }
22
23      public int getAttack() {
24          return attack;
25      }
26
27      public int getHealth() {
28          return health;
29      }
30
31      public int getDefense() {
32          return defense;
33      }
34
35      public String getName() {
36          return name;
37      }
38
39      public void setAttack(int attack) {
40          this.attack = attack;
41      }
42
43      public void setDefense(int defense) {
44          this.defense = defense;
45      }
46
47      public void setHealth(int health) {
48          this.health = health;
49      }
50
51      public void setName(String name) {
52          this.name = name;
53      }
54
55      public Boolean isAlive() {
56          // à compléter
57      }
58
59      public static void checkDead() {
60          // à compléter
61      }
62
63      public static void checkHealth() {
64          // à compléter
65      }
```

```
66
67      public void attack(String type, Fighter other) {
68          // à compléter
69      }
70  }
```

Question 1: (🕓 *5 minutes*) isAlive()
Définir une méthode `isAlive()` de type `boolean` qui retournera `true` si l'instance a plus que 0 points de vie et `false` si l'instance en a moins.

Question 2: (🕓 *10 minutes*) checkDead()
Définir une méthode `checkDead()` qui parcourt la liste des instances, et contrôle que chacune d'entre elle est encore en vie. Si ce n'est pas le cas, l'instance en question est supprimée de la liste des instances et le message "`nomInstance` est mort" sera affiché.

Question 3: (🕓 *5 minutes*) checkHealth()
Définir une méthode `checkHealth()` qui parcourt la liste des instances et affiche le nombre de points de vie qui reste au combattant sous le format "`nomInstance` a encore `healthInstance` points de vie".

Question 4: (🕓 *10 minutes*) attack(String type Fighter other)
Définir une méthode `attack(String type, Fighter other)` qui permettra de retirer des points de vie au combattant `other` en fonction de l'attaque de l'instance appelée, du type d'attaque sélectionné et de la défense de `other`.

Commencez par contrôler si `other` est encore en vie. Si tel n'est pas le cas, indiquez qu'il est déjà mort : "`otherName` est déjà mort".
Si `other` est encore en vie, retirez des points de vie à `other`. Le nombre de points de vie devant être retiré se calcule en utilisant la formule suivante : `attackModifier.get(type) * attackInstance - defenseOther`. Appelez ensuite les fonctions `checkDead()` et `checkHealth()` afin d'avoir un aperçu des combattants restants et de leur santé.

Pour terminer, vous pouvez exécuter le code ci-dessous (disponible dans le dossier Code sur Moodle) pour vérifier que votre programme fonctionne correctement :

```
1  public class Question4 {
2      public static void main(String[] args) {
3          Fighter p1 = new Fighter("P1", 10, 2, 2);
4          Fighter p2 = new Fighter("P2", 10, 2, 2);
5          Fighter p3 =  new Fighter("P3", 10, 2, 2);
6          p1.attack("pied",p2);
7          p1.attack("poing",p2);
8          p1.attack("tete",p2);
9          p1.attack("tete",p2);
10     }
11 }
```

Vous devriez obtenir ce résultat :

```
1  P1 a encore 10 points de vie
2  P2 a encore 8 points de vie
3  P3 a encore 10 points de vie
4  P1 a encore 10 points de vie
5  P2 a encore 8 points de vie
6  P3 a encore 10 points de vie
7  P1 a encore 10 points de vie
8  P2 a encore 4 points de vie
9  P3 a encore 10 points de vie
10 P2 est mort
11 P1 a encore 10 points de vie
12 P3 a encore 10 points de vie
```

```
13
14 Process finished with exit code 0
```

2 Notions d'héritage - Python (30 minutes)

Question 5: (🕓 *30 minutes*) **Un exemple appliqué**
Dans les établissements universitaires, on rencontre souvent des problèmes lors du calcul de salaires du personnel. Sans penser aux recherches effectuées par certains professeurs, on va essayer de calculer les salaires de ceux qui sont reconnus comme 'Professeur' (ordinaire, titulaire, associé ou assistant) à l'université et ceux qui y donnent des cours à temps partiel (on va les considérer comme 'Collaborateurs' dans cet exercice).
La classe mère dans ce cas est nommée `Enseignant`, qui possède une propriété - le salaire annuel moyen. On voudrait que la méthode qui calcule cette quantité renvoie 60 000 (dollars américains) si l'enseignant a moins de 10 ans d'expérience, et 100 000 sinon. Si l'enseignant travaille à temps partiel, la méthode devrait renvoyer une chaîne de caractères qui dit 'Le salaire annuel ne s'applique pas aux collaborateurs'.
Ensuite, on veut calculer la paye mensuelle pour chaque type d'employé. Pour les `Professeurs`, la paye devrait être calculée sur la base de deux sources de revenu : un salaire mensuel et une commission pour chaque comité où ils participent.
D'autre part, pour les `Collaborateurs`, la paye est calculée sur une base horaire i.e *taux horaire* $\times$ *nombres d'heures de travail (par mois)*.
Complétez le code ci-dessous :

```
1  class Enseignant:
2      def __init__(self, name, years_experience, full_time):
3          ...
4      def salaire_annuel_moyen(self):
5          ...
6
7  class Professeur(Enseignant):
8      def __init__(self, name, years_experience, monthly_salary,
       commission, num_committees):
9          ...
10     def paye_mensuelle(self):
11         ...
12
13 class Collaborateur(Enseignant):
14     def __init__(self, name, years_experience, hours_per_month, rate):
15         ...
16     def paye_mensuelle(self):
17         ...
18
19 prof1 = Professeur("Alexandra", 8, 3000, 200, 4)
20 prof2 = Collaborateur("David", 10, 40, 30)
21 # exemples
22 print(prof1.salaire_annuel_moyen())
23 print(prof2.paye_mensuelle())
```